

ARCHITECTURE

Architecture

Component Diagram

```
graph TB
    subgraph DocProcessor
        Loader[pkg/loader]
        Feature[pkg/feature]
        Coverage[pkg/coverage]
        DocGraph[pkg/docgraph]
        LLM[pkg/llm]
        Config[pkg/config]
    end

    Docs[(Project Docs)] --> Loader
    Loader --> Feature
    Feature --> Coverage
    Feature --> DocGraph
    LLM --> Feature
    Config --> Loader
    Config --> Feature
```

Sequence Diagram

```
sequenceDiagram
    participant Caller
    participant Loader
    participant Builder as FeatureMapBuilder
    participant LLM as LLMAgent
    participant Tracker as CoverageTracker

    Caller->>Loader: LoadDir(ctx, path)
    Loader-->>Caller: []Document

    Caller->>Builder: BuildFromDocs(ctx, docs)
    Builder-->>Caller: *FeatureMap
```

```

Caller->>Builder: Enrich(ctx, fm, agent)
Builder->>LLM: InferScreens(ctx, features)
LLM-->>Builder: []ExpectedScreen
Builder->>LLM: GenerateTestSteps(ctx, feature)
LLM-->>Builder: []TestStep
Builder-->>Caller: nil (enriched in-place)

Caller->>Tracker: RegisterFeature(f, platforms)
Caller->>Tracker: MarkVerified(id, platform, evidence)
Caller->>Tracker: Coverage()
Tracker-->>Caller: CoverageReport

```

Class Diagram

```

classDiagram
    class Loader {
        <<interface>>
        +LoadDir(ctx, path) []Document, error
        +LoadFile(ctx, path) Document, error
        +SupportedFormats() []string
    }

    class Document {
        +Path string
        +Format string
        +Title string
        +Content string
        +Sections []Section
        +Links []string
        +ModifiedAt time.Time
    }

    class FeatureMapBuilder {
        <<interface>>
        +BuildFromDocs(ctx, docs) *FeatureMap, error
        +Enrich(ctx, fm, agent) error
        +Merge(maps...) *FeatureMap
    }

    class FeatureMap {
        +Features []Feature
        +Screens []ExpectedScreen
        +Workflows []Workflow
        +Categories map
        +PlatformMatrix map
        +DocGraph *DocGraph
        +AddFeature(f)
        +FeatureByID(id) *Feature
    }

```

```

class CoverageTracker {
    <<interface>>
    +MarkVerified(id, platform, evidence)
    +MarkFailed(id, platform, issue)
    +MarkSkipped(id, platform, reason)
    +Coverage() CoverageReport
    +CoverageByPlatform(platform) float64
    +CoverageByCategory(category) float64
    +Unverified() []Feature
    +Export() CoverageSnapshot
}

class LLMAgent {
    <<interface>>
    +Summarize(ctx, text) string, error
    +ExtractFeatures(ctx, text) []RawFeature, error
    +ClassifyFeature(ctx, feature) FeatureCategory, error
    +InferScreens(ctx, features) []ExpectedScreen, error
    +GenerateTestSteps(ctx, feature) []TestStep, error
}

Loader --> Document
FeatureMapBuilder --> FeatureMap
FeatureMapBuilder --> LLMAgent
FeatureMap --> DocGraph
CoverageTracker --> Feature

```

State Diagram

```

stateDiagram-v2
    [*] --> Unverified: RegisterFeature

    Unverified --> Verified: MarkVerified
    Unverified --> Failed: MarkFailed
    Unverified --> Skipped: MarkSkipped

    Failed --> Verified: MarkVerified (retry)
    Skipped --> Verified: MarkVerified (retry)
    Verified --> Failed: MarkFailed (regression)

```

Flowchart

```

flowchart TD
    A[Start] --> B[Load Documents]
    B -->|Success| C[Parse Markdown/YAML/HTML]
    B -->|Error| Z[Return Error]
    C --> D[Extract Features Heuristic]
    D --> E[LLM Available?]
    E -->|Yes| F[LLM Extract Features]

```

```
E -->|No| G[Use Heuristic Results]
F --> H[Build FeatureMap]
G --> H
H --> I[Enrich with LLM]
I --> J[Register in CoverageTracker]
J --> K[Execute Verification]
K --> L{Feature Status}
L -->|Pass| M[MarkVerified]
L -->|Fail| N[MarkFailed]
L -->|Skip| O[MarkSkipped]
M --> P[Generate Report]
N --> P
O --> P
P --> Q[End]
```

Package Dependencies

```
config (no deps)
docgraph (no deps)
llm (no deps)
loader (yaml.v3)
feature (loader, llm, docgraph)
coverage (no internal deps)
```

Thread Safety

The CoverageTracker implementation uses `sync.RWMutex` for concurrent access:

- Read operations (`Coverage()`, `CoverageByPlatform()`, `Unverified()`) use `RLock()`
- Write operations (`MarkVerified()`, `MarkFailed()`, `MarkSkipped()`) use `Lock()`

The DocGraph also uses `sync.RWMutex` for concurrent node/edge operations.

Design Decisions

1. **LLMAgent is injected** -- No module-level dependency on LLMOrchestrator
2. **Feature IDs are deterministic** -- Same feature name produces same ID across runs
3. **Heuristic fallback** -- Feature extraction works without an LLM agent
4. **Prompt templates versioned** -- Trackable changes to LLM prompts

5. **MaxFileSize limit** -- 10 MB cap prevents OOM on large binary files